user_model.dart

```
class UserModel {
  final String id;
  final String nombre;
  final String correo;
  final String telefono;

  UserModel({
    required this.id,
    required this.nombre,
    required this.correo,
    required this.telefono,
  });

  factory UserModel.fromMap(Map<String, dynamic> data) {
    return UserModel(
      id: data["id_usuario"],
      nombre: data["nombre"],
      correo: data["correo"],
      telefono: data["telefono"],
    );
  }
}
```

admin_page.dart

```
import 'package:flutter/material.dart';
import 'package:eliteform/widgets/users_list.dart';
import '../services/auth_service.dart';
import 'login_page.dart';
import 'admin_pedidos_page.dart';

class AdminPage extends StatelessWidget {
  const AdminPage({super.key});

  Future<void> _logout(BuildContext context) async {
    final authService = AuthService();
    await authService.logout();
    if (context.mounted) {
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(builder: (context) => const LoginPage()),
      );
    }
```

```
  }

  @override
  Widget build(BuildContext context) {
    return DefaultTabController(
      length: 2,
      child: Scaffold(
        appBar: AppBar(
          title: const Text('Panel de Administrador'),
          actions: [
            IconButton(
              icon: const Icon(Icons.logout),
              onPressed: () => _logout(context),
            ),
          ],
          bottom: const TabBar(
            indicatorColor: Colors.orangeAccent,
            labelColor: Colors.orangeAccent,
            unselectedLabelColor: Colors.white54,
            tabs: [
              Tab(
                icon: Icon(Icons.people_outline),
                text: 'Usuarios',
              ),
              Tab(
                icon: Icon(Icons.receipt_long_outlined),
                text: 'Pedidos',
              ),
            ],
          ),
        ),
        body: const TabBarView(
          children: [
            UsersList(),
            AdminPedidosPage(),
          ],
        ),
      ),
    );
  }
}
```

admin_pedidos_page.dart

```
import 'package:flutter/material.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

class AdminPedidosPage extends StatelessWidget {
  const AdminPedidosPage({super.key});

  Future<void> _aceptarPedido(BuildContext context, String pedidoId,
      String uidUsuario, Map<String, dynamic> pedidoData) async {
    final bool confirmar = await showDialog(
          context: context,
          builder: (context) => AlertDialog(
            backgroundColor: const Color(0xFF1E293B),
            title: const Text('Aceptar pedido',
                style: TextStyle(color: Colors.white)),
            content: Text(
              '¿Confirmar pago de ${pedidoData['nombre_usuario']} por
el ${pedidoData['plan']}?',
              style: const TextStyle(color: Colors.white70),
            ),
            actions: [
              TextButton(
                onPressed: () => Navigator.pop(context, false),
                child: const Text('Cancelar',
                    style: TextStyle(color: Colors.white54)),
              ),
              ElevatedButton(
                onPressed: () => Navigator.pop(context, true),
                child: const Text('Aceptar pago'),
              ),
            ],
          ),
        ) ??
        false;

    if (!confirmar) return;

    // Calcular próxima fecha de pago (1 mes desde hoy)
    final DateTime hoy = DateTime.now();
    final DateTime proximoPago = DateTime(hoy.year, hoy.month + 1,
hoy.day);

    final batch = FirebaseFirestore.instance.batch();
```

```
    // 1. Actualizar el pedido a "Aceptado"
    final pedidoRef =
        FirebaseFirestore.instance.collection('pedidos').doc(pedidoId);
    batch.update(pedidoRef, {
      'estado': 'Aceptado',
      'fecha_aceptacion': FieldValue.serverTimestamp(),
      'fecha_proximo_pago': Timestamp.fromDate(proximoPago),
    });

    // 2. Actualizar el documento del usuario con su membresía activa
    final usuarioRef =

FirebaseFirestore.instance.collection('usuarios').doc(uidUsuario);
    batch.update(usuarioRef, {
      'membresia_activa': true,
      'plan_activo': pedidoData['plan'],
      'fecha_proximo_pago': Timestamp.fromDate(proximoPago),
      'pedido_id': pedidoId,
    });

    await batch.commit();

    if (!context.mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content:
            Text('Membresía de ${pedidoData['nombre_usuario']} activada
✓'),
        backgroundColor: Colors.green,
      ),
    );
  }

  Future<void> _rechazarPedido(BuildContext context, String pedidoId,
      String uidUsuario, String nombreUsuario) async {
    final bool confirmar = await showDialog(
          context: context,
          builder: (context) => AlertDialog(
            backgroundColor: const Color(0xFF1E293B),
            title: const Text('Eliminar pedido',
                style: TextStyle(color: Colors.white)),
            content: Text(
```

```
              '¿Eliminar el pedido de $nombreUsuario? El usuario podrá
solicitar un nuevo plan.',
              style: const TextStyle(color: Colors.white70),
            ),
            actions: [
              TextButton(
                onPressed: () => Navigator.pop(context, false),
                child: const Text('Cancelar',
                    style: TextStyle(color: Colors.white54)),
              ),
              ElevatedButton(
                style: ElevatedButton.styleFrom(
                    backgroundColor: Colors.red, foregroundColor:
Colors.white),
                onPressed: () => Navigator.pop(context, true),
                child: const Text('Eliminar'),
              ),
            ],
          ),
        ) ??
        false;

    if (!confirmar) return;

    final batch = FirebaseFirestore.instance.batch();

    // 1. Eliminar el pedido
    batch

.delete(FirebaseFirestore.instance.collection('pedidos').doc(pedidoId))
;

    // 2. Limpiar membresía del usuario si la tenía
    batch.update(

FirebaseFirestore.instance.collection('usuarios').doc(uidUsuario), {
      'membresia_activa': false,
      'plan_activo': FieldValue.delete(),
      'fecha_proximo_pago': FieldValue.delete(),
      'pedido_id': FieldValue.delete(),
    });

    await batch.commit();
```

```
  if (!context.mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('Pedido eliminado'),
      backgroundColor: Colors.red,
    ),
  );
}

Color _colorEstado(String estado) {
  switch (estado) {
    case 'Aceptado':
      return Colors.greenAccent;
    case 'Pendiente':
      return Colors.amberAccent;
    default:
      return Colors.white54;
  }
}

IconData _iconoEstado(String estado) {
  switch (estado) {
    case 'Aceptado':
      return Icons.check_circle;
    case 'Pendiente':
      return Icons.access_time;
    default:
      return Icons.help_outline;
  }
}

@override
Widget build(BuildContext context) {
  return StreamBuilder<QuerySnapshot>(
    stream: FirebaseFirestore.instance
        .collection('pedidos')
        .orderBy('fecha_pedido', descending: true)
        .snapshots(),
    builder: (context, snapshot) {
      if (snapshot.hasError) {
        return const Center(child: Text('Error cargando pedidos'));
      }
```

```
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Center(child: CircularProgressIndicator());
        }
        if (!snapshot.hasData || snapshot.data!.docs.isEmpty) {
          return Center(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: const [
                Icon(Icons.inbox_outlined, color: Colors.white24, size:
64),
                SizedBox(height: 16),
                Text('No hay pedidos aún',
                    style: TextStyle(color: Colors.white54)),
              ],
            ),
          );
        }

        final pedidos = snapshot.data!.docs;
        final pendientes =
            pedidos.where((p) => p['estado'] == 'Pendiente').toList();
        final aceptados =
            pedidos.where((p) => p['estado'] == 'Aceptado').toList();

        return ListView(
          padding: const EdgeInsets.all(16),
          children: [
            // Contador rápido
            Row(
              children: [
                _ContadorChip(
                  label: 'Pendientes',
                  count: pendientes.length,
                  color: Colors.amberAccent,
                ),
                const SizedBox(width: 10),
                _ContadorChip(
                  label: 'Aceptados',
                  count: aceptados.length,
                  color: Colors.greenAccent,
                ),
              ],
            ),
```

```
            const SizedBox(height: 20),

            // Pedidos pendientes primero
            if (pendientes.isNotEmpty) ...[
              const _SeccionHeader(
                  titulo: 'Pendientes de pago',
                  icono: Icons.access_time,
                  color: Colors.amberAccent),
              const SizedBox(height: 10),
              ...pendientes.map((doc) => _TarjetaPedido(
                    pedidoId: doc.id,
                    data: doc.data() as Map<String, dynamic>,
                    onAceptar: (id, uid, data) =>
                        _aceptarPedido(context, id, uid, data),
                    onRechazar: (id, uid, nombre) =>
                        _rechazarPedido(context, id, uid, nombre),
                  )),
              const SizedBox(height: 20),
            ],

            // Pedidos aceptados
            if (aceptados.isNotEmpty) ...[
              const _SeccionHeader(
                  titulo: 'Membresías activas',
                  icono: Icons.check_circle,
                  color: Colors.greenAccent),
              const SizedBox(height: 10),
              ...aceptados.map((doc) => _TarjetaPedido(
                    pedidoId: doc.id,
                    data: doc.data() as Map<String, dynamic>,
                    onAceptar: (id, uid, data) =>
                        _aceptarPedido(context, id, uid, data),
                    onRechazar: (id, uid, nombre) =>
                        _rechazarPedido(context, id, uid, nombre),
                  )),
            ],
          ],
        );
      },
    );
  }
}
```

```
// ─── Widgets auxiliares ──────────────────────────────────────────────

class _ContadorChip extends StatelessWidget {
  final String label;
  final int count;
  final Color color;
  const _ContadorChip(
      {required this.label, required this.count, required this.color});

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      decoration: BoxDecoration(
        color: color.withOpacity(0.12),
        borderRadius: BorderRadius.circular(50),
        border: Border.all(color: color.withOpacity(0.3)),
      ),
      child: Row(
        children: [
          Text(
            '$count',
            style: TextStyle(
                color: color, fontWeight: FontWeight.bold, fontSize:
18),
          ),
          const SizedBox(width: 6),
          Text(label,
              style: TextStyle(color: color.withOpacity(0.8), fontSize:
13)),
        ],
      ),
    );
  }
}

class _SeccionHeader extends StatelessWidget {
  final String titulo;
  final IconData icono;
  final Color color;
  const _SeccionHeader(
```

```
      {required this.titulo, required this.icono, required
this.color});

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Icon(icono, color: color, size: 18),
        const SizedBox(width: 8),
        Text(
          titulo,
          style: TextStyle(
              color: color, fontWeight: FontWeight.bold, fontSize: 15),
        ),
      ],
    );
  }
}

class _TarjetaPedido extends StatelessWidget {
  final String pedidoId;
  final Map<String, dynamic> data;
  final Function(String id, String uid, Map<String, dynamic> data)
onAceptar;
  final Function(String id, String uid, String nombre) onRechazar;

  const _TarjetaPedido({
    required this.pedidoId,
    required this.data,
    required this.onAceptar,
    required this.onRechazar,
  });

  @override
  Widget build(BuildContext context) {
    final String estado = data['estado'] ?? 'Pendiente';
    final bool esAceptado = estado == 'Aceptado';
    final Color colorEstado =
        esAceptado ? Colors.greenAccent : Colors.amberAccent;

    String fechaStr = '—';
    if (data['fecha_pedido'] != null) {
```

```
      final DateTime fecha = (data['fecha_pedido'] as
Timestamp).toDate();
      fechaStr = '${fecha.day}/${fecha.month}/${fecha.year}';
    }

    String proximoPagoStr = '—';
    if (data['fecha_proximo_pago'] != null) {
      final DateTime fp = (data['fecha_proximo_pago'] as
Timestamp).toDate();
      proximoPagoStr = '${fp.day}/${fp.month}/${fp.year}';
    }

    return Container(
      margin: const EdgeInsets.only(bottom: 12),
      decoration: BoxDecoration(
        color: const Color(0xFF1E293B),
        borderRadius: BorderRadius.circular(14),
        border: Border.all(
          color: esAceptado
              ? Colors.greenAccent.withOpacity(0.3)
              : Colors.amberAccent.withOpacity(0.2),
          width: 1,
        ),
      ),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // Header: nombre + estado
            Row(
              children: [
                const CircleAvatar(
                  radius: 18,
                  backgroundColor: Color(0xFF334155),
                  child: Icon(Icons.person, color: Colors.white54,
size: 20),
                ),
                const SizedBox(width: 10),
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
```

```
                    Text(
                      data['nombre_usuario'] ?? 'Usuario',
                      style: const TextStyle(
                          color: Colors.white,
                          fontWeight: FontWeight.bold,
                          fontSize: 15),
                    ),
                    Text(
                      'Pedido: $fechaStr',
                      style: const TextStyle(
                          color: Colors.white54, fontSize: 11),
                    ),
                  ],
                ),
              ),
              Container(
                padding:
                    const EdgeInsets.symmetric(horizontal: 10,
vertical: 4),
                decoration: BoxDecoration(
                  color: colorEstado.withOpacity(0.12),
                  borderRadius: BorderRadius.circular(50),
                  border: Border.all(color:
colorEstado.withOpacity(0.4)),
                ),
                child: Text(
                  estado,
                  style: TextStyle(
                      color: colorEstado,
                      fontSize: 11,
                      fontWeight: FontWeight.bold),
                ),
              ),
            ],
          ),
          const SizedBox(height: 12),
          const Divider(color: Colors.white12, height: 1),
          const SizedBox(height: 12),

          // Info del pedido
          _InfoFila(
              icono: Icons.fitness_center,
              label: 'Plan',
```

```
                valor: data['plan'] ?? '—'),
            const SizedBox(height: 6),
            _InfoFila(
                icono: Icons.attach_money,
                label: 'Monto',
                valor: '\$${data['precio']} MXN',
                valorColor: Colors.orangeAccent),
            const SizedBox(height: 6),
            _InfoFila(
                icono: Icons.payment,
                label: 'Método',
                valor: data['metodo_pago'] ?? '—'),
            if (esAceptado) ...[
              const SizedBox(height: 6),
              _InfoFila(
                  icono: Icons.event,
                  label: 'Próximo pago',
                  valor: proximoPagoStr,
                  valorColor: Colors.greenAccent),
            ],

            const SizedBox(height: 14),

            // Botones de acción
            if (!esAceptado)
              Row(
                children: [
                  Expanded(
                    child: ElevatedButton.icon(
                      onPressed: () =>
                          onAceptar(pedidoId, data['uid_usuario'],
data),
                      icon: const Icon(Icons.check, size: 16),
                      label: const Text('Aceptar pago'),
                      style: ElevatedButton.styleFrom(
                        padding: const EdgeInsets.symmetric(vertical:
10),
                      ),
                    ),
                  ),
                  const SizedBox(width: 10),
                  OutlinedButton.icon(
```

```
                    onPressed: () => onRechazar(pedidoId,
data['uid_usuario'],
                        data['nombre_usuario'] ?? 'usuario'),
                    icon: const Icon(Icons.delete_outline,
                        size: 16, color: Colors.redAccent),
                    label: const Text('Eliminar',
                        style: TextStyle(color: Colors.redAccent)),
                    style: OutlinedButton.styleFrom(
                      side: const BorderSide(color: Colors.redAccent),
                      padding: const EdgeInsets.symmetric(vertical:
10),
                    ),
                  ),
                ],
              ),
            if (esAceptado)
              SizedBox(
                width: double.infinity,
                child: OutlinedButton.icon(
                  onPressed: () => onRechazar(pedidoId,
data['uid_usuario'],
                      data['nombre_usuario'] ?? 'usuario'),
                  icon: const Icon(Icons.cancel_outlined,
                      size: 16, color: Colors.redAccent),
                  label: const Text('Cancelar membresía',
                      style: TextStyle(color: Colors.redAccent)),
                  style: OutlinedButton.styleFrom(
                    side: const BorderSide(color: Colors.redAccent),
                    padding: const EdgeInsets.symmetric(vertical: 10),
                  ),
                ),
              ),
          ],
        ),
      ),
    );
  }
}

class _InfoFila extends StatelessWidget {
  final IconData icono;
  final String label;
  final String valor;
```

```
  final Color? valorColor;
  const _InfoFila(
      {required this.icono,
      required this.label,
      required this.valor,
      this.valorColor});

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Icon(icono, color: Colors.white38, size: 15),
        const SizedBox(width: 8),
        Text(label,
            style: const TextStyle(color: Colors.white54, fontSize:
13)),
        const Spacer(),
        Text(
          valor,
          style: TextStyle(
              color: valorColor ?? Colors.white70,
              fontSize: 13,
              fontWeight: FontWeight.w500),
        ),
      ],
    );
  }
}
```

confirmacion_pago_page.dart

```
import 'package:flutter/material.dart';

class ConfirmacionPagoPage extends StatelessWidget {
  final Map<String, dynamic> plan;
  final String metodoPago;

  const ConfirmacionPagoPage({
    super.key,
    required this.plan,
    required this.metodoPago,
  });

  @override
```

```
Widget build(BuildContext context) {
  return Scaffold(
    body: SafeArea(
      child: Padding(
        padding: const EdgeInsets.all(28),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Spacer(),

            // Ícono de éxito animado
            Container(
              padding: const EdgeInsets.all(24),
              decoration: BoxDecoration(
                color: Colors.green.withOpacity(0.12),
                shape: BoxShape.circle,
                border: Border.all(
                  color: Colors.green.withOpacity(0.4),
                  width: 2,
                ),
              ),
              child: const Icon(
                Icons.check_circle_outline,
                color: Colors.greenAccent,
                size: 72,
              ),
            ),

            const SizedBox(height: 28),

            const Text(
              '¡Pedido confirmado!',
              style: TextStyle(
                fontSize: 28,
                fontWeight: FontWeight.bold,
                color: Colors.white,
              ),
              textAlign: TextAlign.center,
            ),

            const SizedBox(height: 10),

            const Text(
```

```
                'Tu solicitud fue registrada exitosamente. Preséntate
en recepción para completar tu pago y activar tu membresía.',
                style: TextStyle(color: Colors.white54, fontSize: 14),
                textAlign: TextAlign.center,
              ),

              const SizedBox(height: 32),

              // Detalle del pedido
              Container(
                width: double.infinity,
                padding: const EdgeInsets.all(20),
                decoration: BoxDecoration(
                  color: const Color(0xFF1E293B),
                  borderRadius: BorderRadius.circular(16),
                  border: Border.all(color: Colors.white12),
                ),
                child: Column(
                  children: [
                    _FilaDetalle(
                      icono: Icons.fitness_center,
                      label: 'Plan',
                      valor: plan['nombre'],
                    ),
                    const Divider(color: Colors.white12, height: 20),
                    _FilaDetalle(
                      icono: Icons.attach_money,
                      label: 'Total',
                      valor: '\$${plan['precio']} MXN',
                      valorColor: Colors.orangeAccent,
                    ),
                    const Divider(color: Colors.white12, height: 20),
                    _FilaDetalle(
                      icono: Icons.payment,
                      label: 'Método de pago',
                      valor: metodoPago,
                    ),
                    const Divider(color: Colors.white12, height: 20),
                    _FilaDetalle(
                      icono: Icons.info_outline,
                      label: 'Estado',
                      valor: 'Pendiente de pago',
                      valorColor: Colors.amberAccent,
```

```
                    ),
                  ],
                ),
              ),

              const SizedBox(height: 16),

              // Nota según método de pago
              Container(
                padding: const EdgeInsets.all(14),
                decoration: BoxDecoration(
                  color: Colors.blueAccent.withOpacity(0.08),
                  borderRadius: BorderRadius.circular(12),
                  border: Border.all(color:
Colors.blueAccent.withOpacity(0.2)),
                ),
                child: Row(
                  children: [
                    const Icon(Icons.info_outline,
                        color: Colors.blueAccent, size: 18),
                    const SizedBox(width: 10),
                    Expanded(
                      child: Text(
                        metodoPago == 'Efectivo'
                            ? 'Acércate a recepción y muestra este
pedido para completar tu pago.'
                            : metodoPago == 'Transferencia'
                                ? 'Solicita los datos de transferencia
en recepción y envía tu comprobante.'
                                : 'Preséntate en recepción con tu
tarjeta para completar el cobro.',
                        style: const TextStyle(
                            color: Colors.white54, fontSize: 12),
                      ),
                    ),
                  ],
                ),
              ),

              const Spacer(),

              // Botón de volver — cierra todas las pantallas apiladas
```

```
              // (confirmacion → resumen → detalle) y regresa al
HomePage
              SizedBox(
                width: double.infinity,
                child: ElevatedButton.icon(
                  onPressed: () {
                    // Elimina todas las rutas hasta llegar al
HomePage,
                    // que ya contiene PlanesPage en su
BottomNavigationBar.
                    // PlanesPage detectará el pedido y mostrará el
estado correcto.
                    Navigator.of(context).popUntil((route) =>
route.isFirst);
                  },
                  icon: const Icon(Icons.home),
                  label: const Text('Volver a los planes'),
                ),
              ),

              const SizedBox(height: 12),
            ],
          ),
        ),
      ),
    );
  }
}

// Widget auxiliar para las filas del resumen
class _FilaDetalle extends StatelessWidget {
  final IconData icono;
  final String label;
  final String valor;
  final Color? valorColor;

  const _FilaDetalle({
    required this.icono,
    required this.label,
    required this.valor,
    this.valorColor,
  });
```

```
  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Icon(icono, color: Colors.orangeAccent, size: 18),
        const SizedBox(width: 10),
        Text(
          label,
          style: const TextStyle(color: Colors.white54, fontSize: 13),
        ),
        const Spacer(),
        Text(
          valor,
          style: TextStyle(
            color: valorColor ?? Colors.white,
            fontWeight: FontWeight.bold,
            fontSize: 13,
          ),
        ),
      ],
    );
  }
}
```

detalle_plan_page.dart

```
import 'package:flutter/material.dart';
import 'resumen_pago_page.dart';

class DetallePlanPage extends StatelessWidget {
  final Map<String, dynamic> plan;
  const DetallePlanPage({super.key, required this.plan});

  @override
  Widget build(BuildContext context) {
    final List<String> beneficios = plan['beneficios'] as List<String>;

    return Scaffold(
      appBar: AppBar(
        title: Text(plan['nombre']),
      ),
      body: SingleChildScrollView(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
```

```
          children: [
            // Banner superior del plan
            Container(
              width: double.infinity,
              padding: const EdgeInsets.symmetric(vertical: 36,
horizontal: 24),
              decoration: BoxDecoration(
                gradient: LinearGradient(
                  colors: [
                    (plan['color'] as Color).withOpacity(0.4),
                    const Color(0xFF0F172A),
                  ],
                  begin: Alignment.topCenter,
                  end: Alignment.bottomCenter,
                ),
              ),
              child: Column(
                children: [
                  Container(
                    padding: const EdgeInsets.all(18),
                    decoration: BoxDecoration(
                      color: (plan['color'] as Color).withOpacity(0.2),
                      shape: BoxShape.circle,
                      border: Border.all(
                        color: plan['color'] as Color,
                        width: 2,
                      ),
                    ),
                    child: Icon(
                      plan['icono'] as IconData,
                      color: plan['color'] as Color,
                      size: 48,
                    ),
                  ),
                  const SizedBox(height: 16),
                  Text(
                    plan['nombre'],
                    style: const TextStyle(
                      fontSize: 26,
                      fontWeight: FontWeight.bold,
                      color: Colors.white,
                    ),
                  ),
```

```
                  const SizedBox(height: 6),
                  Text(
                    plan['descripcion'],
                    textAlign: TextAlign.center,
                    style: const TextStyle(color: Colors.white60,
fontSize: 14),
                  ),
                  const SizedBox(height: 20),
                  // Precio grande
                  Container(
                    padding: const EdgeInsets.symmetric(
                        horizontal: 28, vertical: 12),
                    decoration: BoxDecoration(
                      color: Colors.orangeAccent.withOpacity(0.15),
                      borderRadius: BorderRadius.circular(50),
                      border: Border.all(
                          color: Colors.orangeAccent.withOpacity(0.4)),
                    ),
                    child: Row(
                      mainAxisSize: MainAxisSize.min,
                      crossAxisAlignment: CrossAxisAlignment.end,
                      children: [
                        Text(
                          '\$${plan['precio']}',
                          style: const TextStyle(
                            fontSize: 38,
                            fontWeight: FontWeight.bold,
                            color: Colors.orangeAccent,
                          ),
                        ),
                        const Padding(
                          padding: EdgeInsets.only(bottom: 6, left: 4),
                          child: Text(
                            '/ mes',
                            style: TextStyle(
                                color: Colors.white54, fontSize: 15),
                          ),
                        ),
                      ],
                    ),
                  ),
                ],
              ),
```

```
            ),

            // Lista de beneficios completa
            Padding(
              padding: const EdgeInsets.all(20),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  const Text(
                    'Lo que incluye este plan',
                    style: TextStyle(
                      fontSize: 18,
                      fontWeight: FontWeight.bold,
                      color: Colors.white,
                    ),
                  ),
                  const SizedBox(height: 16),

                  Container(
                    padding: const EdgeInsets.all(16),
                    decoration: BoxDecoration(
                      color: const Color(0xFF1E293B),
                      borderRadius: BorderRadius.circular(16),
                    ),
                    child: Column(
                      children: beneficios.map((b) {
                        return Padding(
                          padding: const EdgeInsets.only(bottom: 14),
                          child: Row(
                            children: [
                              Container(
                                padding: const EdgeInsets.all(4),
                                decoration: BoxDecoration(
                                  color:

Colors.orangeAccent.withOpacity(0.15),
                                  shape: BoxShape.circle,
                                ),
                                child: const Icon(
                                  Icons.check,
                                  color: Colors.orangeAccent,
                                  size: 16,
                                ),
```

```
                              ),
                              const SizedBox(width: 12),
                              Expanded(
                                child: Text(
                                  b,
                                  style: const TextStyle(
                                    color: Colors.white70,
                                    fontSize: 14,
                                  ),
                                ),
                              ),
                            ],
                          ),
                        );
                      }).toList(),
                    ),
                  ),

                  const SizedBox(height: 24),

                  // Nota informativa
                  Container(
                    padding: const EdgeInsets.all(14),
                    decoration: BoxDecoration(
                      color: Colors.blueAccent.withOpacity(0.08),
                      borderRadius: BorderRadius.circular(12),
                      border: Border.all(
                          color: Colors.blueAccent.withOpacity(0.2)),
                    ),
                    child: Row(
                      children: const [
                        Icon(Icons.info_outline,
                            color: Colors.blueAccent, size: 18),
                        SizedBox(width: 10),
                        Expanded(
                          child: Text(
                            'La membresía se activa el día en que
realizas el pago. Puedes cancelar en cualquier momento desde tu
perfil.',
                            style: TextStyle(
                                color: Colors.white54, fontSize: 12),
                          ),
                        ),
```

```
                ],
              ),
            ),

            const SizedBox(height: 30),

            // Botón de contratar
            SizedBox(
              width: double.infinity,
              child: ElevatedButton.icon(
                onPressed: () {
                  Navigator.push(
                    context,
                    MaterialPageRoute(
                      builder: (context) =>
                          ResumenPagoPage(plan: plan),
                    ),
                  );
                },
                icon: const Icon(Icons.shopping_cart_checkout),
                label: Text('Contratar ${plan['nombre']}'),
              ),
            ),

            const SizedBox(height: 12),

            // Botón de cancelar
            SizedBox(
              width: double.infinity,
              child: TextButton(
                onPressed: () => Navigator.pop(context),
                child: const Text(
                  'Volver a los planes',
                  style: TextStyle(color: Colors.white54),
                ),
              ),
            ),
          ],
        ),
      ),
    ],
  ),
),
```

```
    );
  }
}
```

dieta_page.dart

```
import 'package:flutter/material.dart';

class DietaPage extends StatelessWidget {
  const DietaPage({super.key});

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Text("Dietas próximamente"),
    );
  }
}
```

ejercicio_page.dart

```
import 'package:flutter/material.dart';

class EjerciciosPage extends StatelessWidget {
  const EjerciciosPage({super.key});

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Text("Ejercicios próximamente"),
    );
  }
}
```

eventos_page.dart

```
import 'package:flutter/material.dart';

class EventosPage extends StatelessWidget {
  const EventosPage({super.key});

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Text("Eventos próximamente"),
    );
```

```
  }
}
```

home_page.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'login_page.dart';
import 'perfil_page.dart';
import 'rutina_page.dart';
import 'dieta_page.dart';
import 'ejercicios_page.dart';
import 'eventos_page.dart';
import 'planes_page.dart';

class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  int _selectedIndex = 0;

  final List<Widget> _pages = const [
    RutinaPage(),
    DietaPage(),
    EjerciciosPage(),
    EventosPage(),
    PlanesPage(),
    PerfilPage(),
  ];

  void _onItemTapped(int index) {
    setState(() {
      _selectedIndex = index;
    });
  }

  Future logout() async {
    await FirebaseAuth.instance.signOut();

    Navigator.pushReplacement(
```

```
    context,
    MaterialPageRoute(builder: (context) => const LoginPage()),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text("EliteForm"),
      actions: [
        IconButton(
          icon: const Icon(Icons.logout),
          onPressed: logout,
        )
      ],
    ),
    body: _pages[_selectedIndex],
    bottomNavigationBar: BottomNavigationBar(
      backgroundColor: const Color(0xFF020617),
      selectedItemColor: Colors.orangeAccent,
      unselectedItemColor: Colors.white54,
      currentIndex: _selectedIndex,
      onTap: _onItemTapped,
      items: const [
        BottomNavigationBarItem(
          icon: Icon(Icons.fitness_center),
          label: "Rutina",
        ),
        BottomNavigationBarItem(
          icon: Icon(Icons.restaurant),
          label: "Dieta",
        ),
        BottomNavigationBarItem(
          icon: Icon(Icons.sports_gymnastics),
          label: "Ejercicios",
        ),
        BottomNavigationBarItem(
          icon: Icon(Icons.event),
          label: "Eventos",
        ),
        BottomNavigationBarItem(
          icon: Icon(Icons.card_membership),
```

```
            label: "Planes",
          ),
          BottomNavigationBarItem(
            icon: Icon(Icons.person),
            label: "Perfil",
          ),
        ],
      ),
    );
  }
}
```

login_page.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

import 'register_page.dart';
import 'home_page.dart';
import '../widgets/app_background.dart';
import 'admin_page.dart';

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final emailController = TextEditingController();
  final passwordController = TextEditingController();

  final adminEmail = "admin@admin.com";
  final adminPassword = "123456";

  Future loginUser() async {
    if (emailController.text.trim() == adminEmail &&
        passwordController.text.trim() == adminPassword) {
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(builder: (context) => const AdminPage()),
      );
```

```
    return;
  }

  try {
    await FirebaseAuth.instance.signInWithEmailAndPassword(
      email: emailController.text.trim(),
      password: passwordController.text.trim(),
    );

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (context) => const HomePage()),
    );
  } on FirebaseAuthException catch (e) {
    showError(e.message);
  }
}

void showError(String? message) {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text("Error"),
      content: Text(message ?? "Credenciales incorrectas"),
    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: AppBackground(
      child: Center(
        child: SingleChildScrollView(
          child: Padding(
            padding: const EdgeInsets.all(25),
            child: Column(
              children: [
                const Icon(
                  Icons.fitness_center,
                  size: 90,
                  color: Colors.orangeAccent,
                ),
```

```
                const SizedBox(height: 20),
                const Text(
                  "Elite Form",
                  style: TextStyle(
                    fontSize: 34,
                    fontWeight: FontWeight.bold,
                    letterSpacing: 2,
                  ),
                ),
                const SizedBox(height: 5),
                const Text(
                  "Entrena. Mejora. Supera.",
                  style: TextStyle(color: Colors.white54),
                ),
                const SizedBox(height: 40),
                Container(
                  padding: const EdgeInsets.all(20),
                  decoration: BoxDecoration(
                    color: const Color(0xFF1E293B),
                    borderRadius: BorderRadius.circular(20),
                  ),
                  child: Column(
                    children: [
                      TextField(
                        controller: emailController,
                        decoration: const InputDecoration(
                          labelText: "Correo",
                          prefixIcon: Icon(Icons.email),
                        ),
                      ),
                      const SizedBox(height: 15),
                      TextField(
                        controller: passwordController,
                        obscureText: true,
                        decoration: const InputDecoration(
                          labelText: "Contraseña",
                          prefixIcon: Icon(Icons.lock),
                        ),
                      ),
                      const SizedBox(height: 25),
                      SizedBox(
                        width: double.infinity,
                        child: ElevatedButton(
```

```
                              onPressed: loginUser,
                              style: ElevatedButton.styleFrom(
                                backgroundColor: Colors.orangeAccent,
                                foregroundColor: Colors.black,
                                padding: const
EdgeInsets.symmetric(vertical: 16),
                                shape: RoundedRectangleBorder(
                                  borderRadius:
BorderRadius.circular(12),
                                ),
                              ),
                              child: const Text(
                                "INICIAR ENTRENAMIENTO",
                                style: TextStyle(fontWeight:
FontWeight.bold),
                              ),
                            ),
                          ),
                          const SizedBox(height: 10),
                          TextButton(
                            onPressed: () {
                              Navigator.push(
                                context,
                                MaterialPageRoute(
                                  builder: (context) => const
RegisterPage(),
                                ),
                              );
                            },
                            child: const Text("Crear cuenta"),
                          )
                        ],
                      ),
                    )
                  ],
                ),
              ),
            ),
          ),
        ),
      );
    }
}
```

perfil_page.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter/services.dart';
import 'login_page.dart';

class PerfilPage extends StatefulWidget {
  const PerfilPage({super.key});

  @override
  State<PerfilPage> createState() => _PerfilPageState();
}

class _PerfilPageState extends State<PerfilPage> {
  final nombreController = TextEditingController();
  final telefonoController = TextEditingController();
  final emailController = TextEditingController();

  bool editando = false;
  bool loading = true;

  Future cargarDatos() async {
    final user = FirebaseAuth.instance.currentUser;

    final doc = await FirebaseFirestore.instance
        .collection("usuarios")
        .doc(user!.uid)
        .get();

    nombreController.text = doc["nombre"];
    telefonoController.text = doc["telefono"];
    emailController.text = user.email ?? "";
    setState(() {
      loading = false;
    });
  }

  Future actualizarPerfil() async {
    if (telefonoController.text.length != 10) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
```

```
            content: Text("El teléfono debe tener exactamente 10
dígitos")),
      );
      return; // Detiene la ejecución
    }
    // -----------------------

    final user = FirebaseAuth.instance.currentUser;

    await FirebaseFirestore.instance
        .collection("usuarios")
        .doc(user!.uid)
        .update({
      "nombre": nombreController.text.trim(),
      "telefono": telefonoController.text.trim(),
    });

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text("Perfil actualizado")),
    );
  }

  Future eliminarCuenta() async {
    final user = FirebaseAuth.instance.currentUser;

    bool confirmar = await showDialog(
          context: context,
          builder: (context) => AlertDialog(
            title: const Text("Eliminar cuenta"),
            content: const Text(
                "Esta acción eliminará tu cuenta permanentemente.
¿Continuar?"),
            actions: [
              TextButton(
                onPressed: () => Navigator.pop(context, false),
                child: const Text("Cancelar"),
              ),
              TextButton(
                onPressed: () => Navigator.pop(context, true),
                child: const Text(
                  "Eliminar",
                  style: TextStyle(color: Colors.red),
                ),
```

```
            ),
          ],
        ),
      ) ??
      false;

  if (!confirmar) return;

  try {
    await FirebaseFirestore.instance
        .collection("usuarios")
        .doc(user!.uid)
        .delete();

    await user.delete();

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (context) => const LoginPage()),
    );
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text("No se pudo eliminar la cuenta")),
    );
  }
}

@override
void initState() {
  super.initState();
  cargarDatos();
}

@override
Widget build(BuildContext context) {
  if (loading) {
    return const Center(child: CircularProgressIndicator());
  }

  final user = FirebaseAuth.instance.currentUser;

  return Padding(
    padding: const EdgeInsets.all(20),
```

```
      child: SingleChildScrollView(
        child: Column(
          children: [
            const Icon(
              Icons.person,
              size: 100,
              color: Colors.orangeAccent,
            ),
            const SizedBox(height: 20),
            if (!editando) ...[
              ListTile(
                leading: const Icon(Icons.badge),
                title: const Text("Nombre"),
                subtitle: Text(nombreController.text),
              ),
              const SizedBox(height: 10),
              ListTile(
                leading: const Icon(Icons.phone),
                title: const Text("Teléfono"),
                subtitle: Text(telefonoController.text),
              ),
              const SizedBox(height: 10),
              ListTile(
                leading: const Icon(Icons.email),
                title: const Text("Correo"),
                subtitle: Text(user!.email ?? ""),
              ),
              const SizedBox(height: 30),
              SizedBox(
                width: double.infinity,
                child: ElevatedButton(
                  onPressed: () {
                    setState(() {
                      editando = true;
                    });
                  },
                  child: const Text("Editar perfil"),
                ),
              ),
              const SizedBox(height: 15),
              SizedBox(
                width: double.infinity,
                child: ElevatedButton(
```

```
                onPressed: eliminarCuenta,
                style: ElevatedButton.styleFrom(
                  backgroundColor: Colors.red,
                  foregroundColor: Colors.white,
                ),
                child: const Text("Eliminar cuenta"),
              ),
            ),
          ],
          if (editando) ...[
            TextField(
              controller: nombreController,
              decoration: const InputDecoration(
                labelText: "Nombre",
                prefixIcon: Icon(Icons.badge),
              ),
            ),
            const SizedBox(height: 15),
            TextField(
              controller: telefonoController,
              keyboardType: TextInputType.number,
              maxLength: 10,
              inputFormatters:
[FilteringTextInputFormatter.digitsOnly],
              decoration: const InputDecoration(
                labelText: "Teléfono",
                prefixIcon: Icon(Icons.phone),
                counterText:
                    "", // Esto oculta el contador "0/10" para no
alterar diseño
              ),
            ),
            const SizedBox(height: 15),
            TextField(
              controller: emailController,
              enabled: false, // Mantiene el campo bloqueado
              decoration: const InputDecoration(
                labelText: "Correo electrónico",
                prefixIcon: Icon(Icons.email),
              ),
            ),
            const SizedBox(height: 30),
            SizedBox(
```

```
                width: double.infinity,
                child: ElevatedButton(
                  onPressed: () async {
                    await actualizarPerfil();

                    setState(() {
                      editando = false;
                    });
                  },
                  child: const Text("Guardar cambios"),
                ),
              ),
              TextButton(
                onPressed: () {
                  setState(() {
                    editando = false;
                  });
                },
                child: const Text("Cancelar"),
              ),
            ],
          ],
        ),
      ),
    );
  }
}
```

planes_page.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'detalle_plan_page.dart';

class PlanesPage extends StatelessWidget {
  const PlanesPage({super.key});

  static const List<Map<String, dynamic>> planes = [
    {
      'nombre': 'Plan Básico',
      'precio': 299,
      'duracion': '1 mes',
      'icono': Icons.fitness_center,
```

```
      'color': Color(0xFF334155),
      'descripcion': 'Ideal para comenzar tu transformación.',
      'beneficios': [
        'Acceso al gym de lunes a viernes',
        'Horario: 6am - 8pm',
        'Uso de máquinas cardiovasculares',
        'Uso de área de pesas',
        'Casillero incluido',
      ],
    },
    {
      'nombre': 'Plan Pro',
      'precio': 499,
      'duracion': '1 mes',
      'icono': Icons.bolt,
      'color': Color(0xFFEA580C),
      'descripcion': 'El más popular. Acceso completo y clases
grupales.',
      'beneficios': [
        'Acceso ilimitado todos los días',
        'Horario: 5am - 10pm',
        'Incluye clases grupales (yoga, spinning, crossfit)',
        'Uso de todas las áreas',
        'Casillero incluido',
        '1 sesión de evaluación física',
      ],
    },
    {
      'nombre': 'Plan Elite',
      'precio': 799,
      'duracion': '1 mes',
      'icono': Icons.workspace_premium,
      'color': Color(0xFFD97706),
      'descripcion': 'La experiencia completa con entrenador
personal.',
      'beneficios': [
        'Acceso ilimitado todos los días',
        'Horario: 24 horas',
        'Clases grupales ilimitadas',
        'Uso de todas las áreas',
        'Casillero premium',
        '4 sesiones con entrenador personal',
        'Plan de nutrición personalizado',
```

```
        'Acceso a área de spa y sauna',
      ],
    },
  ];

  @override
  Widget build(BuildContext context) {
    final user = FirebaseAuth.instance.currentUser;
    if (user == null) return const SizedBox();

    return StreamBuilder<DocumentSnapshot>(
      stream: FirebaseFirestore.instance
          .collection('usuarios')
          .doc(user.uid)
          .snapshots(),
      builder: (context, userSnap) {
        if (userSnap.connectionState == ConnectionState.waiting) {
          return const Center(child: CircularProgressIndicator());
        }

        final userData = userSnap.data?.data() as Map<String, dynamic>?
?? {};
        final bool membresiaActiva = userData['membresia_activa'] ==
true;
        final String? pedidoId = userData['pedido_id'];

        // ESTADO 3: Membresía activa
        if (membresiaActiva && pedidoId != null) {
          return _VistaMembresia(
            userData: userData,
            pedidoId: pedidoId,
            uid: user.uid,
          );
        }

        // ESTADO 2: Pedido pendiente
        if (!membresiaActiva && pedidoId != null) {
          return StreamBuilder<DocumentSnapshot>(
            stream: FirebaseFirestore.instance
                .collection('pedidos')
                .doc(pedidoId)
                .snapshots(),
            builder: (context, pedidoSnap) {
```

```
              if (pedidoSnap.connectionState ==
ConnectionState.waiting) {
                return const Center(child:
CircularProgressIndicator());
              }
              if (!pedidoSnap.hasData || !pedidoSnap.data!.exists) {
                return _VistaPlanes(uid: user.uid);
              }
              final pedidoData =
                  pedidoSnap.data!.data() as Map<String, dynamic>;
              return _VistaPedidoPendiente(
                pedidoId: pedidoId,
                pedidoData: pedidoData,
                uid: user.uid,
              );
            },
          );
        }

        // ESTADO 1: Sin pedido
        return _VistaPlanes(uid: user.uid);
      },
    );
  }
}

//
_____________________________________________________________________
_____
// ESTADO 1: Catálogo de planes
//
_____________________________________________________________________
_____
class _VistaPlanes extends StatelessWidget {
  final String uid;
  const _VistaPlanes({required this.uid});

  @override
  Widget build(BuildContext context) {
    return SingleChildScrollView(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
```

```
        children: [
          Container(
            width: double.infinity,
            padding: const EdgeInsets.all(20),
            decoration: BoxDecoration(
              gradient: const LinearGradient(
                colors: [Color(0xFF1E293B), Color(0xFF0F172A)],
                begin: Alignment.topLeft,
                end: Alignment.bottomRight,
              ),
              borderRadius: BorderRadius.circular(16),
              border: Border.all(
                  color: Colors.orangeAccent.withOpacity(0.3), width:
1),
            ),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: const [
                Icon(Icons.star, color: Colors.orangeAccent, size: 32),
                SizedBox(height: 8),
                Text(
                  'Elige tu membresía',
                  style: TextStyle(
                      fontSize: 22,
                      fontWeight: FontWeight.bold,
                      color: Colors.white),
                ),
                SizedBox(height: 4),
                Text(
                  'Selecciona el plan que mejor se adapte a tus
objetivos.',
                  style: TextStyle(color: Colors.white54, fontSize:
13),
                ),
              ],
            ),
          ),
          const SizedBox(height: 20),
          ...PlanesPage.planes.map((plan) => _PlanCard(plan:
plan)).toList(),
        ],
      ),
    );
```

```
  }
}

class _PlanCard extends StatelessWidget {
  final Map<String, dynamic> plan;
  const _PlanCard({required this.plan});

  @override
  Widget build(BuildContext context) {
    final bool esPopular = plan['nombre'] == 'Plan Pro';
    return Container(
      margin: const EdgeInsets.only(bottom: 16),
      decoration: BoxDecoration(
        color: const Color(0xFF1E293B),
        borderRadius: BorderRadius.circular(16),
        border: esPopular
            ? Border.all(color: Colors.orangeAccent, width: 2)
            : Border.all(color: Colors.white12, width: 1),
      ),
      child: Column(
        children: [
          if (esPopular)
            Container(
              width: double.infinity,
              padding: const EdgeInsets.symmetric(vertical: 6),
              decoration: const BoxDecoration(
                color: Colors.orangeAccent,
                borderRadius: BorderRadius.only(
                  topLeft: Radius.circular(14),
                  topRight: Radius.circular(14),
                ),
              ),
              child: const Center(
                child: Text('⭐ MÁS POPULAR',
                    style: TextStyle(
                        color: Colors.black,
                        fontWeight: FontWeight.bold,
                        fontSize: 12,
                        letterSpacing: 1)),
              ),
            ),
          Padding(
            padding: const EdgeInsets.all(20),
```

```
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Row(
                  children: [
                    Container(
                      padding: const EdgeInsets.all(10),
                      decoration: BoxDecoration(
                        color: (plan['color'] as
Color).withOpacity(0.2),
                        borderRadius: BorderRadius.circular(12),
                      ),
                      child: Icon(plan['icono'] as IconData,
                          color: plan['color'] as Color, size: 26),
                    ),
                    const SizedBox(width: 12),
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          Text(plan['nombre'],
                              style: const TextStyle(
                                  fontSize: 18,
                                  fontWeight: FontWeight.bold,
                                  color: Colors.white)),
                          Text(plan['descripcion'],
                              style: const TextStyle(
                                  fontSize: 12, color:
Colors.white54)),
                        ],
                      ),
                    ),
                  ],
                ),
                const SizedBox(height: 16),
                Row(
                  crossAxisAlignment: CrossAxisAlignment.end,
                  children: [
                    Text('\$${plan['precio']}',
                        style: const TextStyle(
                            fontSize: 32,
                            fontWeight: FontWeight.bold,
                            color: Colors.orangeAccent)),
```

```
                    const SizedBox(width: 4),
                    const Padding(
                      padding: EdgeInsets.only(bottom: 5),
                      child: Text('/ mes',
                          style:
                              TextStyle(color: Colors.white54,
fontSize: 14)),
                    ),
                  ],
                ),
                const SizedBox(height: 16),
                ...((plan['beneficios'] as List<String>)
                    .take(3)
                    .map((b) => Padding(
                          padding: const EdgeInsets.only(bottom: 6),
                          child: Row(
                            children: [
                              const Icon(Icons.check_circle,
                                  color: Colors.orangeAccent, size:
16),
                              const SizedBox(width: 8),
                              Expanded(
                                  child: Text(b,
                                      style: const TextStyle(
                                          color: Colors.white70,
                                          fontSize: 13))),
                            ],
                          ),
                        ))),
                if ((plan['beneficios'] as List<String>).length > 3)
                  Padding(
                    padding: const EdgeInsets.only(top: 2, bottom: 8),
                    child: Text(
                        '+ ${(plan['beneficios'] as
List<String>).length - 3} beneficios más...',
                        style: const TextStyle(
                            color: Colors.orangeAccent, fontSize: 12)),
                  ),
                const SizedBox(height: 8),
                SizedBox(
                  width: double.infinity,
                  child: ElevatedButton(
                    onPressed: () => Navigator.push(
```

```
                        context,
                        MaterialPageRoute(
                            builder: (context) => DetallePlanPage(plan:
plan))),
                    child: const Text('Ver plan completo'),
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

//
_____________________________________________________________________________
_____
// ESTADO 2: Pedido pendiente
//
_____________________________________________________________________________
_____
class _VistaPedidoPendiente extends StatelessWidget {
  final String pedidoId;
  final Map<String, dynamic> pedidoData;
  final String uid;
  const _VistaPedidoPendiente(
      {required this.pedidoId, required this.pedidoData, required
this.uid});

  Future<void> _cancelarPedido(BuildContext context) async {
    final bool confirmar = await showDialog(
          context: context,
          builder: (ctx) => AlertDialog(
            backgroundColor: const Color(0xFF1E293B),
            title: const Text('Cancelar pedido',
                style: TextStyle(color: Colors.white)),
            content: const Text(
                '¿Quieres cancelar este pedido? Podrás solicitar un
nuevo plan.',
                style: TextStyle(color: Colors.white70)),
            actions: [
```

```
              TextButton(
                  onPressed: () => Navigator.pop(ctx, false),
                  child: const Text('No',
                      style: TextStyle(color: Colors.white54))),
              ElevatedButton(
                  style: ElevatedButton.styleFrom(
                      backgroundColor: Colors.red,
                      foregroundColor: Colors.white),
                  onPressed: () => Navigator.pop(ctx, true),
                  child: const Text('Sí, cancelar')),
            ],
          ),
        ) ??
        false;

    if (!confirmar) return;

    final batch = FirebaseFirestore.instance.batch();
    batch

.delete(FirebaseFirestore.instance.collection('pedidos').doc(pedidoId))
;

batch.update(FirebaseFirestore.instance.collection('usuarios').doc(uid)
, {
      'pedido_id': FieldValue.delete(),
      'membresia_activa': false,
    });
    await batch.commit();

    if (!context.mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
          content: Text('Pedido cancelado. Ya puedes elegir otro
plan.')),
    );
  }

  @override
  Widget build(BuildContext context) {
    String fechaStr = '—';
    if (pedidoData['fecha_pedido'] != null) {
```

```
      final DateTime f = (pedidoData['fecha_pedido'] as
Timestamp).toDate();
      fechaStr = '${f.day}/${f.month}/${f.year}';
    }

    return SingleChildScrollView(
      padding: const EdgeInsets.all(20),
      child: Column(
        children: [
          const SizedBox(height: 10),
          Container(
            width: double.infinity,
            padding: const EdgeInsets.all(20),
            decoration: BoxDecoration(
              color: Colors.amberAccent.withOpacity(0.08),
              borderRadius: BorderRadius.circular(16),
              border: Border.all(color:
Colors.amberAccent.withOpacity(0.3)),
            ),
            child: Column(
              children: const [
                Icon(Icons.access_time, color: Colors.amberAccent,
size: 48),
                SizedBox(height: 12),
                Text(
                  'Pedido en revisión',
                  style: TextStyle(
                      fontSize: 20,
                      fontWeight: FontWeight.bold,
                      color: Colors.white),
                ),
                SizedBox(height: 6),
                Text(
                  'Tu pedido fue recibido. El administrador verificará
tu pago y activará tu membresía pronto.',
                  textAlign: TextAlign.center,
                  style: TextStyle(color: Colors.white54, fontSize:
13),
                ),
              ],
            ),
          ),
          const SizedBox(height: 24),
```

```
        Container(
          width: double.infinity,
          padding: const EdgeInsets.all(18),
          decoration: BoxDecoration(
            color: const Color(0xFF1E293B),
            borderRadius: BorderRadius.circular(14),
          ),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const Text('Detalle de tu pedido',
                  style: TextStyle(
                      color: Colors.white,
                      fontWeight: FontWeight.bold,
                      fontSize: 15)),
              const SizedBox(height: 14),
              _FilaPedido(
                  icono: Icons.fitness_center,
                  label: 'Plan',
                  valor: pedidoData['plan'] ?? '—'),
              const Divider(color: Colors.white12, height: 20),
              _FilaPedido(
                  icono: Icons.attach_money,
                  label: 'Monto',
                  valor: '\$${pedidoData['precio']} MXN',
                  valorColor: Colors.orangeAccent),
              const Divider(color: Colors.white12, height: 20),
              _FilaPedido(
                  icono: Icons.payment,
                  label: 'Método de pago',
                  valor: pedidoData['metodo_pago'] ?? '—'),
              const Divider(color: Colors.white12, height: 20),
              _FilaPedido(
                  icono: Icons.calendar_today,
                  label: 'Fecha del pedido',
                  valor: fechaStr),
              const Divider(color: Colors.white12, height: 20),
              const _FilaPedido(
                  icono: Icons.info_outline,
                  label: 'Estado',
                  valor: 'Pendiente de confirmación',
                  valorColor: Colors.amberAccent),
            ],
```

```
            ),
          ),
          const SizedBox(height: 16),
          Container(
            padding: const EdgeInsets.all(14),
            decoration: BoxDecoration(
              color: Colors.blueAccent.withOpacity(0.08),
              borderRadius: BorderRadius.circular(12),
              border: Border.all(color:
Colors.blueAccent.withOpacity(0.2)),
            ),
            child: Row(
              children: [
                const Icon(Icons.info_outline,
                    color: Colors.blueAccent, size: 18),
                const SizedBox(width: 10),
                Expanded(
                  child: Text(
                    pedidoData['metodo_pago'] == 'Efectivo'
                        ? 'Acércate a recepción con este pedido para
realizar tu pago en efectivo.'
                        : pedidoData['metodo_pago'] == 'Transferencia'
                            ? 'Solicita los datos bancarios en
recepción y envía tu comprobante.'
                            : 'Preséntate en recepción con tu tarjeta
para que procesen el cobro.',
                    style: const TextStyle(color: Colors.white54,
fontSize: 12),
                  ),
                ),
              ],
            ),
          ),
          const SizedBox(height: 28),
          SizedBox(
            width: double.infinity,
            child: OutlinedButton.icon(
              onPressed: () => _cancelarPedido(context),
              icon: const Icon(Icons.cancel_outlined, color:
Colors.redAccent),
              label: const Text('Cancelar pedido',
                  style: TextStyle(color: Colors.redAccent)),
              style: OutlinedButton.styleFrom(
```

```
                side: const BorderSide(color: Colors.redAccent),
                padding: const EdgeInsets.symmetric(vertical: 14),
              ),
            ),
          ),
        ],
      ),
    );
  }
}

//
_____________________________________________________________________________
_____
// ESTADO 3: Membresía activa
//
_____________________________________________________________________________
_____
class _VistaMembresia extends StatelessWidget {
  final Map<String, dynamic> userData;
  final String pedidoId;
  final String uid;

  const _VistaMembresia(
      {required this.userData, required this.pedidoId, required
this.uid});

  Future<void> _cancelarMembresia(BuildContext context) async {
    final bool confirmar = await showDialog(
          context: context,
          builder: (ctx) => AlertDialog(
            backgroundColor: const Color(0xFF1E293B),
            title: const Text('Cancelar membresía',
                style: TextStyle(color: Colors.white)),
            content: const Text(
                '¿Estás seguro? Perderás el acceso al gym y podrás
contratar un nuevo plan cuando quieras.',
                style: TextStyle(color: Colors.white70)),
            actions: [
              TextButton(
                  onPressed: () => Navigator.pop(ctx, false),
                  child: const Text('No, conservar',
                      style: TextStyle(color: Colors.white54))),
```

```
              ElevatedButton(
                  style: ElevatedButton.styleFrom(
                      backgroundColor: Colors.red,
                      foregroundColor: Colors.white),
                  onPressed: () => Navigator.pop(ctx, true),
                  child: const Text('Sí, cancelar')),
            ],
          ),
        ) ??
        false;

    if (!confirmar) return;

    final batch = FirebaseFirestore.instance.batch();
    batch

.delete(FirebaseFirestore.instance.collection('pedidos').doc(pedidoId))
;

batch.update(FirebaseFirestore.instance.collection('usuarios').doc(uid)
, {
      'membresia_activa': false,
      'plan_activo': FieldValue.delete(),
      'fecha_proximo_pago': FieldValue.delete(),
      'pedido_id': FieldValue.delete(),
    });
    await batch.commit();

    if (!context.mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
          content: Text('Membresía cancelada.'), backgroundColor:
Colors.red),
    );
  }

  String _formatearFecha(dynamic timestamp) {
    if (timestamp == null) return '—';
    final DateTime f = (timestamp as Timestamp).toDate();
    const meses = [
      '',
      'enero',
      'febrero',
```

```
    'marzo',
    'abril',
    'mayo',
    'junio',
    'julio',
    'agosto',
    'septiembre',
    'octubre',
    'noviembre',
    'diciembre'
  ];
  return '${f.day} de ${meses[f.month]} de ${f.year}';
}

int _diasRestantes(dynamic timestamp) {
  if (timestamp == null) return 0;
  final DateTime fp = (timestamp as Timestamp).toDate();
  return fp.difference(DateTime.now()).inDays.clamp(0, 999);
}

@override
Widget build(BuildContext context) {
  final String planActivo = userData['plan_activo'] ?? 'Plan';
  final dynamic proximoPagoTs = userData['fecha_proximo_pago'];
  final int diasRestantes = _diasRestantes(proximoPagoTs);

  final Map<String, dynamic> planInfo = PlanesPage.planes.firstWhere(
      (p) => p['nombre'] == planActivo,
      orElse: () => PlanesPage.planes[0]);

  return SingleChildScrollView(
    padding: const EdgeInsets.all(20),
    child: Column(
      children: [
        // Tarjeta principal membresía
        Container(
          width: double.infinity,
          padding: const EdgeInsets.all(24),
          decoration: BoxDecoration(
            gradient: LinearGradient(
              colors: [
                (planInfo['color'] as Color).withOpacity(0.3),
                const Color(0xFF1E293B),
```

```
                ],
                begin: Alignment.topLeft,
                end: Alignment.bottomRight,
              ),
              borderRadius: BorderRadius.circular(20),
              border: Border.all(
                  color: Colors.greenAccent.withOpacity(0.4), width:
2),
            ),
            child: Column(
              children: [
                Row(
                  mainAxisAlignment: MainAxisAlignment.spaceBetween,
                  children: [
                    Container(
                      padding: const EdgeInsets.symmetric(
                          horizontal: 12, vertical: 6),
                      decoration: BoxDecoration(
                        color: Colors.greenAccent.withOpacity(0.15),
                        borderRadius: BorderRadius.circular(50),
                        border: Border.all(
                            color:
Colors.greenAccent.withOpacity(0.4)),
                      ),
                      child: Row(
                        children: const [
                          Icon(Icons.check_circle,
                              color: Colors.greenAccent, size: 14),
                          SizedBox(width: 6),
                          Text('ACTIVA',
                              style: TextStyle(
                                  color: Colors.greenAccent,
                                  fontSize: 11,
                                  fontWeight: FontWeight.bold,
                                  letterSpacing: 1)),
                        ],
                      ),
                    ),
                    Text('EliteForm',
                        style: TextStyle(
                            color: Colors.white.withOpacity(0.4),
                            fontSize: 12,
                            letterSpacing: 2)),
```

```
                  ],
                ),
                const SizedBox(height: 20),
                Icon(planInfo['icono'] as IconData,
                    color: planInfo['color'] as Color, size: 52),
                const SizedBox(height: 12),
                Text(planActivo,
                    style: const TextStyle(
                        fontSize: 26,
                        fontWeight: FontWeight.bold,
                        color: Colors.white)),
                const SizedBox(height: 4),
                Text(planInfo['descripcion'],
                    textAlign: TextAlign.center,
                    style:
                        const TextStyle(color: Colors.white54,
fontSize: 13)),
                const SizedBox(height: 20),
                Container(
                  padding:
                      const EdgeInsets.symmetric(horizontal: 20,
vertical: 12),
                  decoration: BoxDecoration(
                    color: Colors.black26,
                    borderRadius: BorderRadius.circular(12),
                  ),
                  child: Row(
                    mainAxisAlignment: MainAxisAlignment.spaceAround,
                    children: [
                      _StatMembresia(
                          valor: '\$${planInfo['precio']}',
                          label: 'Mensualidad',
                          color: Colors.orangeAccent),
                      Container(width: 1, height: 36, color:
Colors.white12),
                      _StatMembresia(
                          valor: '$diasRestantes',
                          label: 'Días restantes',
                          color: diasRestantes < 7
                              ? Colors.redAccent
                              : Colors.greenAccent),
                    ],
                  ),
```

```
                ),
              ],
            ),
          ),

          const SizedBox(height: 20),

          // Próximo pago
          Container(
            width: double.infinity,
            padding: const EdgeInsets.all(18),
            decoration: BoxDecoration(
              color: const Color(0xFF1E293B),
              borderRadius: BorderRadius.circular(14),
            ),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                const Text('Información de membresía',
                    style: TextStyle(
                        color: Colors.white,
                        fontWeight: FontWeight.bold,
                        fontSize: 15)),
                const SizedBox(height: 14),
                _FilaPedido(
                    icono: Icons.event_repeat,
                    label: 'Próximo pago',
                    valor: _formatearFecha(proximoPagoTs),
                    valorColor:
                        diasRestantes < 7 ? Colors.redAccent :
Colors.white),
                const Divider(color: Colors.white12, height: 20),
                _FilaPedido(
                    icono: Icons.attach_money,
                    label: 'Monto a pagar',
                    valor: '\$${planInfo['precio']} MXN',
                    valorColor: Colors.orangeAccent),
              ],
            ),
          ),

          const SizedBox(height: 16),
```

```
          // Beneficios activos
          Container(
            width: double.infinity,
            padding: const EdgeInsets.all(18),
            decoration: BoxDecoration(
              color: const Color(0xFF1E293B),
              borderRadius: BorderRadius.circular(14),
            ),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                const Text('Tu plan incluye',
                    style: TextStyle(
                        color: Colors.white,
                        fontWeight: FontWeight.bold,
                        fontSize: 15)),
                const SizedBox(height: 12),
                ...(planInfo['beneficios'] as List<String>).map((b) =>
Padding(
                      padding: const EdgeInsets.only(bottom: 8),
                      child: Row(
                        children: [
                          Container(
                            padding: const EdgeInsets.all(3),
                            decoration: BoxDecoration(
                              color:
Colors.greenAccent.withOpacity(0.15),
                              shape: BoxShape.circle,
                            ),
                            child: const Icon(Icons.check,
                                color: Colors.greenAccent, size: 13),
                          ),
                          const SizedBox(width: 10),
                          Expanded(
                              child: Text(b,
                                  style: const TextStyle(
                                      color: Colors.white70, fontSize:
13))),
                        ],
                      ),
                    )),
              ],
            ),
```

```
          ),

          const SizedBox(height: 24),

          // Cancelar membresía
          SizedBox(
            width: double.infinity,
            child: OutlinedButton.icon(
              onPressed: () => _cancelarMembresia(context),
              icon: const Icon(Icons.cancel_outlined, color:
Colors.redAccent),
              label: const Text('Cancelar membresía',
                  style: TextStyle(color: Colors.redAccent)),
              style: OutlinedButton.styleFrom(
                side: const BorderSide(color: Colors.redAccent),
                padding: const EdgeInsets.symmetric(vertical: 14),
              ),
            ),
          ),

          const SizedBox(height: 8),
          const Text(
            'Al cancelar perderás el acceso al gym. Podrás contratar un
nuevo plan cuando quieras.',
            textAlign: TextAlign.center,
            style: TextStyle(color: Colors.white24, fontSize: 11),
          ),
          const SizedBox(height: 20),
        ],
      ),
    );
  }
}

// ─── Widgets compartidos
──────────────────────────────────────────────────────────

class _FilaPedido extends StatelessWidget {
  final IconData icono;
  final String label;
  final String valor;
  final Color? valorColor;
  const _FilaPedido(
```

```
      {required this.icono,
      required this.label,
      required this.valor,
      this.valorColor});

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Icon(icono, color: Colors.white38, size: 16),
        const SizedBox(width: 10),
        Text(label,
            style: const TextStyle(color: Colors.white54, fontSize:
13)),
        const Spacer(),
        Text(valor,
            style: TextStyle(
                color: valorColor ?? Colors.white70,
                fontSize: 13,
                fontWeight: FontWeight.w500)),
      ],
    );
  }
}

class _StatMembresia extends StatelessWidget {
  final String valor;
  final String label;
  final Color color;
  const _StatMembresia(
      {required this.valor, required this.label, required this.color});

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text(valor,
            style: TextStyle(
                color: color, fontSize: 20, fontWeight:
FontWeight.bold)),
        const SizedBox(height: 2),
        Text(label,
```

```
            style: const TextStyle(color: Colors.white54, fontSize:
11)),
      ],
    );
  }
}
```

register_page.dart

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart'; // [cite: 1] Necesario para
FilteringTextInputFormatter
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'home_page.dart';

class RegisterPage extends StatefulWidget {
  const RegisterPage({super.key});

  @override
  State<RegisterPage> createState() => _RegisterPageState();
}

class _RegisterPageState extends State<RegisterPage> {
  final emailController = TextEditingController();
  final passwordController = TextEditingController();
  final nombreController = TextEditingController();
  final telefonoController = TextEditingController();

  Future registerUser() async {
    // [cite: 2] VALIDACIÓN LÓGICA: Verifica que tenga exactamente 10
dígitos antes de llamar a Firebase
    if (telefonoController.text.length != 10) {
      showError("El teléfono debe tener exactamente 10 dígitos");
      return;
    }

    try {
      UserCredential userCredential =
          await FirebaseAuth.instance.createUserWithEmailAndPassword(
        email: emailController.text.trim(),
        password: passwordController.text.trim(),
      );
```

```
      await FirebaseFirestore.instance
          .collection("usuarios")
          .doc(userCredential.user!.uid)
          .set({
        "id_usuario": userCredential.user!.uid,
        "nombre": nombreController.text.trim(),
        "correo": emailController.text.trim(),
        "telefono": telefonoController.text.trim(),
        "fecha_registro": FieldValue.serverTimestamp(),
      });

      if (mounted) {
        Navigator.pushReplacement(
          context,
          MaterialPageRoute(builder: (context) => const HomePage()),
        );
      }
    } on FirebaseAuthException catch (e) {
      showError(e.message);
    }
  }

  void showError(String? message) {
    showDialog(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text("Error"),
        content: Text(message ?? "Ocurrió un error"),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("Registro"),
      ),
      body: Padding(
        padding: const EdgeInsets.all(20),
        child: SingleChildScrollView(
          // [cite: 3] Recomendado para evitar errores de overflow con
el teclado
```

```
          child: Column(
            children: [
              TextField(
                controller: nombreController,
                decoration: const InputDecoration(labelText: "Nombre"),
              ),
              const SizedBox(height: 10),

              // [cite: 4] CAMBIOS EN EL CAMPO DE TELÉFONO
              TextField(
                controller: telefonoController,
                decoration: const InputDecoration(
                  labelText: "Teléfono",
                  hintText: "Ej: 1234567890",
                  counterText: "", // Oculta el contador visual si lo
deseas
                ),
                keyboardType: TextInputType
                    .number, // [cite: 5] Muestra el teclado numérico
                maxLength: 10, // [cite: 6] Limita la escritura a 10
caracteres
                inputFormatters: [
                  FilteringTextInputFormatter
                      .digitsOnly, // [cite: 7] Bloquea letras y
símbolos
                ],
              ),

              const SizedBox(height: 10),
              TextField(
                controller: emailController,
                decoration: const InputDecoration(labelText: "Email"),
              ),
              const SizedBox(height: 10),
              TextField(
                controller: passwordController,
                decoration: const InputDecoration(labelText:
"Contraseña"),
                obscureText: true,
              ),
              const SizedBox(height: 20),
              ElevatedButton(
                onPressed: registerUser,
```

```
                child: const Text("Registrarse"),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

resumen_pago_page.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'confirmacion_pago_page.dart';

class ResumenPagoPage extends StatefulWidget {
  final Map<String, dynamic> plan;
  const ResumenPagoPage({super.key, required this.plan});

  @override
  State<ResumenPagoPage> createState() => _ResumenPagoPageState();
}

class _ResumenPagoPageState extends State<ResumenPagoPage> {
  // Método de pago seleccionado
  String _metodoPago = 'Efectivo';
  bool _procesando = false;
  String _nombreUsuario = '';

  final List<Map<String, dynamic>> _metodos = [
    {
      'label': 'Efectivo',
      'icono': Icons.payments_outlined,
      'descripcion': 'Paga directamente en recepción',
    },
    {
      'label': 'Transferencia',
      'icono': Icons.account_balance,
      'descripcion': 'Transferencia bancaria o SPEI',
    },
    {
      'label': 'Tarjeta',
```

```
    'icono': Icons.credit_card,
    'descripcion': 'Crédito o débito en recepción',
  },
];

@override
void initState() {
  super.initState();
  _cargarNombreUsuario();
}

Future<void> _cargarNombreUsuario() async {
  final user = FirebaseAuth.instance.currentUser;
  if (user == null) return;
  final doc = await FirebaseFirestore.instance
      .collection('usuarios')
      .doc(user.uid)
      .get();
  if (doc.exists) {
    setState(() {
      _nombreUsuario = doc['nombre'] ?? '';
    });
  }
}

Future<void> _confirmarPedido() async {
  setState(() => _procesando = true);

  try {
    final user = FirebaseAuth.instance.currentUser;
    if (user == null) return;

    // Guardar el pedido en Firestore
    await FirebaseFirestore.instance.collection('pedidos').add({
      'uid_usuario': user.uid,
      'nombre_usuario': _nombreUsuario,
      'plan': widget.plan['nombre'],
      'precio': widget.plan['precio'],
      'metodo_pago': _metodoPago,
      'estado': 'Pendiente',
      'fecha_pedido': FieldValue.serverTimestamp(),
    });
```

```
      if (!context.mounted) return;

      // Navegar a confirmación
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(
          builder: (context) => ConfirmacionPagoPage(
            plan: widget.plan,
            metodoPago: _metodoPago,
          ),
        ),
      );
    } catch (e) {
      setState(() => _procesando = false);
      if (!context.mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Error al procesar el pedido.
Intenta de nuevo.')),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Resumen de pago'),
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // Resumen del plan elegido
            const Text(
              'Plan seleccionado',
              style: TextStyle(
                fontSize: 16,
                fontWeight: FontWeight.bold,
                color: Colors.white,
              ),
            ),
            const SizedBox(height: 10),
```

```
            Container(
              width: double.infinity,
              padding: const EdgeInsets.all(16),
              decoration: BoxDecoration(
                color: const Color(0xFF1E293B),
                borderRadius: BorderRadius.circular(14),
                border:
                    Border.all(color:
Colors.orangeAccent.withOpacity(0.4)),
              ),
              child: Row(
                children: [
                  Container(
                    padding: const EdgeInsets.all(10),
                    decoration: BoxDecoration(
                      color: (widget.plan['color'] as
Color).withOpacity(0.2),
                      borderRadius: BorderRadius.circular(10),
                    ),
                    child: Icon(
                      widget.plan['icono'] as IconData,
                      color: widget.plan['color'] as Color,
                      size: 28,
                    ),
                  ),
                  const SizedBox(width: 14),
                  Expanded(
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        Text(
                          widget.plan['nombre'],
                          style: const TextStyle(
                            fontSize: 16,
                            fontWeight: FontWeight.bold,
                            color: Colors.white,
                          ),
                        ),
                        Text(
                          widget.plan['duracion'],
                          style: const TextStyle(
                              color: Colors.white54, fontSize: 13),
                        ),
```

```
                ],
              ),
            ),
            Text(
              '\$${widget.plan['precio']}',
              style: const TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
                color: Colors.orangeAccent,
              ),
            ),
          ],
        ),
      ),

      const SizedBox(height: 28),

      // Datos del usuario
      const Text(
        'Titular de la membresía',
        style: TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.bold,
          color: Colors.white,
        ),
      ),
      const SizedBox(height: 10),
      Container(
        width: double.infinity,
        padding: const EdgeInsets.all(16),
        decoration: BoxDecoration(
          color: const Color(0xFF1E293B),
          borderRadius: BorderRadius.circular(14),
        ),
        child: Row(
          children: [
            const Icon(Icons.person, color: Colors.orangeAccent),
            const SizedBox(width: 12),
            Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
```

```
                        _nombreUsuario.isEmpty ? 'Cargando...' :
_nombreUsuario,
                        style: const TextStyle(
                          color: Colors.white,
                          fontWeight: FontWeight.bold,
                          fontSize: 15,
                        ),
                      ),
                      Text(
                        FirebaseAuth.instance.currentUser?.email ?? '',
                        style: const TextStyle(
                            color: Colors.white54, fontSize: 12),
                      ),
                    ],
                  ),
                ],
              ),
            ),

            const SizedBox(height: 28),

            // Método de pago
            const Text(
              'Método de pago',
              style: TextStyle(
                fontSize: 16,
                fontWeight: FontWeight.bold,
                color: Colors.white,
              ),
            ),
            const SizedBox(height: 10),

            ..._metodos.map((metodo) {
              final bool seleccionado = _metodoPago == metodo['label'];
              return GestureDetector(
                onTap: () {
                  setState(() => _metodoPago = metodo['label']);
                },
                child: Container(
                  margin: const EdgeInsets.only(bottom: 10),
                  padding: const EdgeInsets.all(14),
                  decoration: BoxDecoration(
                    color: seleccionado
```

```
                ? Colors.orangeAccent.withOpacity(0.12)
                : const Color(0xFF1E293B),
            borderRadius: BorderRadius.circular(12),
            border: Border.all(
              color: seleccionado
                  ? Colors.orangeAccent
                  : Colors.white12,
              width: seleccionado ? 2 : 1,
            ),
          ),
          child: Row(
            children: [
              Icon(
                metodo['icono'] as IconData,
                color: seleccionado
                    ? Colors.orangeAccent
                    : Colors.white54,
                size: 24,
              ),
              const SizedBox(width: 14),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      metodo['label'],
                      style: TextStyle(
                        color: seleccionado
                            ? Colors.orangeAccent
                            : Colors.white,
                        fontWeight: FontWeight.bold,
                        fontSize: 14,
                      ),
                    ),
                    Text(
                      metodo['descripcion'],
                      style: const TextStyle(
                          color: Colors.white54, fontSize: 12),
                    ),
                  ],
                ),
              ),
              if (seleccionado)
```

```
                const Icon(Icons.check_circle,
                    color: Colors.orangeAccent, size: 20),
            ],
          ),
        ),
      );
    }).toList(),

    const SizedBox(height: 28),

    // Total final
    Container(
      width: double.infinity,
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
        color: const Color(0xFF1E293B),
        borderRadius: BorderRadius.circular(14),
      ),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          const Text(
            'Total a pagar',
            style: TextStyle(
              color: Colors.white,
              fontSize: 16,
              fontWeight: FontWeight.bold,
            ),
          ),
          Text(
            '\$${widget.plan['precio']} MXN',
            style: const TextStyle(
              color: Colors.orangeAccent,
              fontSize: 22,
              fontWeight: FontWeight.bold,
            ),
          ),
        ],
      ),
    ),

    const SizedBox(height: 24),
```

```
            // Botón de confirmar
            SizedBox(
              width: double.infinity,
              child: ElevatedButton.icon(
                onPressed: _procesando ? null : _confirmarPedido,
                icon: _procesando
                    ? const SizedBox(
                        width: 18,
                        height: 18,
                        child: CircularProgressIndicator(
                          strokeWidth: 2,
                          color: Colors.black,
                        ),
                      )
                    : const Icon(Icons.check),
                label: Text(
                    _procesando ? 'Procesando...' : 'Confirmar
pedido'),
              ),
            ),

            const SizedBox(height: 12),

            SizedBox(
              width: double.infinity,
              child: TextButton(
                onPressed: () => Navigator.pop(context),
                child: const Text(
                  'Regresar',
                  style: TextStyle(color: Colors.white54),
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

rutina_page.dart

```
import 'package:flutter/material.dart';
```

```
class RutinaPage extends StatelessWidget {
  const RutinaPage({super.key});

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Text("Rutinas próximamente"),
    );
  }
}
```

auth_service.dart

```
import 'package:firebase_auth/firebase_auth.dart';

class AuthService {
  final FirebaseAuth _auth = FirebaseAuth.instance;

  Future<UserCredential> register(String email, String password) async
{
    return await _auth.createUserWithEmailAndPassword(
      email: email,
      password: password,
    );
  }

  Future<UserCredential> login(String email, String password) async {
    return await _auth.signInWithEmailAndPassword(
      email: email,
      password: password,
    );
  }

  Future logout() async {
    await _auth.signOut();
  }

  User? get currentUser => _auth.currentUser;
}
```

firestore_service.dart

```
import 'package:cloud_firestore/cloud_firestore.dart';

class FirestoreService {
```

```
  final FirebaseFirestore _db = FirebaseFirestore.instance;

  Future createUser({
    required String uid,
    required String nombre,
    required String correo,
    required String telefono,
  }) async {
    await _db.collection("usuarios").doc(uid).set({
      "id_usuario": uid,
      "nombre": nombre,
      "correo": correo,
      "telefono": telefono,
      "fecha_registro": FieldValue.serverTimestamp(),
    });
  }

  Future updateUser(uid, nombre, telefono) async {
    await _db.collection("usuarios").doc(uid).update({
      "nombre": nombre,
      "telefono": telefono,
    });
  }

  Future deleteUser(uid) async {
    await _db.collection("usuarios").doc(uid).delete();
  }

  Stream<QuerySnapshot> getUsers() {
    return _db.collection("usuarios").snapshots();
  }
}
```

app_background.dart

```
import 'package:flutter/material.dart';

class AppBackground extends StatelessWidget {
  final Widget child;

  const AppBackground({super.key, required this.child});

  @override
  Widget build(BuildContext context) {
```

```
    return Container(
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          colors: [Color(0xFF0F172A), Color(0xFF020617)],
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
        ),
      ),
      child: child,
    );
  }
}
```

users_list.dart

```
import 'package:flutter/material.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

class UsersList extends StatelessWidget {
  const UsersList({super.key});

  @override
  Widget build(BuildContext context) {
    return StreamBuilder<QuerySnapshot>(
      stream:
FirebaseFirestore.instance.collection("usuarios").snapshots(),
      builder: (context, snapshot) {
        if (snapshot.hasError) {
          return const Center(
            child: Text("Error cargando usuarios"),
          );
        }

        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Center(
            child: CircularProgressIndicator(),
          );
        }

        if (!snapshot.hasData || snapshot.data!.docs.isEmpty) {
          return const Center(
            child: Text("No hay usuarios registrados"),
          );
        }
```

```
        final users = snapshot.data!.docs;

        return ListView.builder(
          itemCount: users.length,
          itemBuilder: (context, index) {
            var user = users[index];

            return Card(
              color: const Color(0xFF1E293B),
              margin: const EdgeInsets.all(10),
              child: ListTile(
                leading: const CircleAvatar(
                  backgroundColor: Colors.transparent,
                  child: Icon(Icons.person, color: Colors.black),
                ),
                title: Text(
                  user["correo"],
                  style: const TextStyle(
                    fontWeight: FontWeight.bold,
                  ),
                ),
                subtitle: Text(user["id_usuario"]),
              ),
            );
          },
        );
      },
    );
  }
}
```

firebase_options.dart

```
// File generated by FlutterFire CLI.
// ignore_for_file: type=lint
import 'package:firebase_core/firebase_core.dart' show FirebaseOptions;
import 'package:flutter/foundation.dart'
    show defaultTargetPlatform, kIsWeb, TargetPlatform;

/// Default [FirebaseOptions] for use with your Firebase apps.
///
/// Example:
/// ```dart
```

```
/// import 'firebase_options.dart';
/// // ...
/// await Firebase.initializeApp(
///   options: DefaultFirebaseOptions.currentPlatform,
/// );
/// ```
class DefaultFirebaseOptions {
  static FirebaseOptions get currentPlatform {
    if (kIsWeb) {
      return web;
    }
    switch (defaultTargetPlatform) {
      case TargetPlatform.android:
        return android;
      case TargetPlatform.iOS:
        return ios;
      case TargetPlatform.macOS:
        return macos;
      case TargetPlatform.windows:
        return windows;
      case TargetPlatform.linux:
        throw UnsupportedError(
          'DefaultFirebaseOptions have not been configured for linux -
'
          'you can reconfigure this by running the FlutterFire CLI
again.',
        );
      default:
        throw UnsupportedError(
          'DefaultFirebaseOptions are not supported for this
platform.',
        );
    }
  }

  static const FirebaseOptions web = FirebaseOptions(
    apiKey: 'AIzaSyBbRQ6uzjMi_7OpcJI6f0ZJEvBd8xT7IsY',
    appId: '1:939544369119:web:4c614d815a576122ba166a',
    messagingSenderId: '939544369119',
    projectId: 'eliteform-9f2fc',
    authDomain: 'eliteform-9f2fc.firebaseapp.com',
    storageBucket: 'eliteform-9f2fc.firebasestorage.app',
    measurementId: 'G-PV4P008WB0',
```

```
  );

  static const FirebaseOptions android = FirebaseOptions(
    apiKey: 'AIzaSyA4gZ8Y7QCemSJHvyRubS0BFtKWI8HOZhA',
    appId: '1:939544369119:android:12ca6edee9257e84ba166a',
    messagingSenderId: '939544369119',
    projectId: 'eliteform-9f2fc',
    storageBucket: 'eliteform-9f2fc.firebasestorage.app',
  );

  static const FirebaseOptions ios = FirebaseOptions(
    apiKey: 'AIzaSyDKvG8A61w-lNq1qJFcCnm3QQPTWuEp9uc',
    appId: '1:939544369119:ios:aec1d68a173efec5ba166a',
    messagingSenderId: '939544369119',
    projectId: 'eliteform-9f2fc',
    storageBucket: 'eliteform-9f2fc.firebasestorage.app',
    iosClientId:
'939544369119-igo0ocpal0gd4aidfg6v0g667g9s73q0.apps.googleusercontent.c
om',
    iosBundleId: 'com.example.eliteform',
  );

  static const FirebaseOptions macos = FirebaseOptions(
    apiKey: 'AIzaSyDKvG8A61w-lNq1qJFcCnm3QQPTWuEp9uc',
    appId: '1:939544369119:ios:aec1d68a173efec5ba166a',
    messagingSenderId: '939544369119',
    projectId: 'eliteform-9f2fc',
    storageBucket: 'eliteform-9f2fc.firebasestorage.app',
    iosClientId:
'939544369119-igo0ocpal0gd4aidfg6v0g667g9s73q0.apps.googleusercontent.c
om',
    iosBundleId: 'com.example.eliteform',
  );

  static const FirebaseOptions windows = FirebaseOptions(
    apiKey: 'AIzaSyBbRQ6uzjMi_7OpcJI6f0ZJEvBd8xT7IsY',
    appId: '1:939544369119:web:f70ca6ab1734f229ba166a',
    messagingSenderId: '939544369119',
    projectId: 'eliteform-9f2fc',
    authDomain: 'eliteform-9f2fc.firebaseapp.com',
    storageBucket: 'eliteform-9f2fc.firebasestorage.app',
    measurementId: 'G-XN7E1GMDHQ',
  );
```

```
}
```

main.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';
import 'pages/login_page.dart';

void main() async {
  // Asegura que los bindings de Flutter estén listos antes de
inicializar Firebase [cite: 396]
  WidgetsFlutterBinding.ensureInitialized();

  // Inicialización de Firebase usando las opciones configuradas por
plataforma [cite: 396]
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: "EliteForm",

      // Configuración centralizada del estilo visual de la aplicación
[cite: 400]
      theme: ThemeData(
        brightness: Brightness.dark,
        scaffoldBackgroundColor:
            const Color(0xFF0F172A), // Fondo azul oscuro profundo
[cite: 400]

        colorScheme: const ColorScheme.dark(
          primary: Colors.orangeAccent,
          secondary: Colors.orangeAccent,
          surface: Color(
```

```
              0xFF1E293B), // Color para tarjetas y contenedores [cite:
401]
        ),

        // Estilo global para la barra superior [cite: 401]
        appBarTheme: const AppBarTheme(
          backgroundColor: Color(0xFF020617),
          foregroundColor: Colors.orangeAccent,
          elevation: 0,
          iconTheme: IconThemeData(color: Colors.orangeAccent),
        ),

        // Configuración predeterminada para botones elevados [cite:
401]
        elevatedButtonTheme: ElevatedButtonThemeData(
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.orangeAccent,
            foregroundColor: Colors.black,
            padding: const EdgeInsets.symmetric(horizontal: 30,
vertical: 15),
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(12),
            ),
          ),
        ),

        // Estilo para los campos de texto (Inputs) [cite: 401]
        inputDecorationTheme: const InputDecorationTheme(
          filled: true,
          fillColor: Color(0xFF1E293B),
          border: OutlineInputBorder(
            borderRadius: BorderRadius.all(Radius.circular(12)),
            borderSide: BorderSide.none,
          ),
          prefixIconColor: Colors.orangeAccent,
        ),

        progressIndicatorTheme: const ProgressIndicatorThemeData(
          color: Colors.orangeAccent,
        ),
      ),

      // Punto de entrada: Pantalla de inicio de sesión [cite: 401]
```

```
      home: const LoginPage(),
    );
  }
}
```